

Search...

메뉴바를 클릭하면 계정을 관리할 수 있어요

Deep Learning

[딥러닝 파이토치 교과서] #01. 머신러닝과 딥러닝

carchive-lab 2026. 9. 7. 21:53

1. 머신러닝 VS 딥러닝

- 머신러닝 : 데이터 → 특성추출(사람) → 분류(컴퓨터) → 결과
- 딥러닝 : 데이터 → 특성추출+분류(컴퓨터) → 결과

구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행한다.	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다.
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다.	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다.
데이터	일반적으로 수천 개의 데이터가 필요하다.	수백만 개 이상의 데이터가 필요하다.
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

2. 머신러닝

2-0. 용어 정리

- 데이터 : 학습 모델을 만들 때 사용되며, 실크제 데이터의 특징이 잘 반영되고 편향되지 않은 데이터셋을 확보해야 한다.

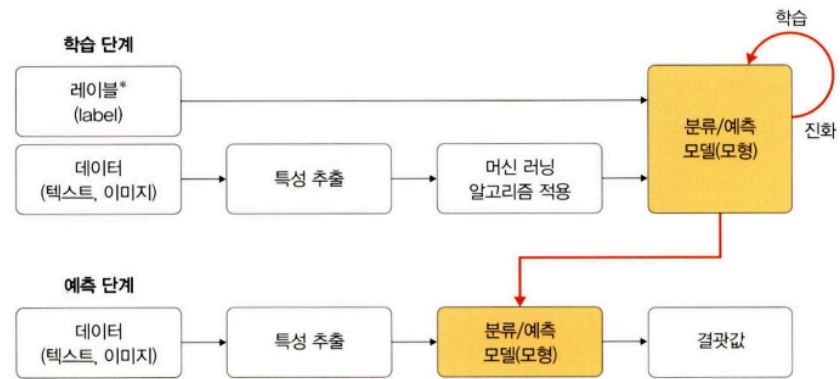
: 데이터셋은 80%는 **훈련 데이터셋**, 20%는 **테스트 데이터셋**으로 나누며, 모델 성능 평가를 위해 훈련 데이터셋을 **검증 데이터셋**으로 일부 떼어낸다. (즉 훈련 데이터의 양이 많지 않을 때는 추천하지 않음) **검증 데이터셋**은 1) 테스트 데이터에 대해 성능을 검증할 수 있고, 2) 훈련 데이터에 대해 과적합이 일어났을 경우를 확인하기 위해 사용된다.

- 모델 : '입력 데이터의 패턴은 @@와 같다'처럼 학습 단계에서 얻은 최종 결과물을 이르며, 가설이라고도 볼 수 있다.

: 학습 단계에서는 '모델 선택 → 모델 학습 및 평가 → 평가를 바탕으로 모델 업데이트'의 과정을 반복하며, 최종적으로 완성된 모델을 문제에 적용해서 분류 및 예측 결과를 도출한다.

- 특성 추출(feature extraction) : 입력 받은 데이터 별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환한다. 머신러닝 단계에서 사람이 직접 수행한다.

2-1. 학습 과정 : 학습 단계 + 예측 단계



2-2. 학습 알고리즘 : 지도 학습, 비지도 학습, 강화학습

- 지도 학습 : 레이블 有
- 비지도 학습 : 레이블 無, 따라서 clustering으로 예측한다.
- 강화학습 : 분류할 수 있는 데이터가 있는 것도 아니고, 데이터가 있더라도 정답이 없는 경우에 사용된다. 오로지 보상이 커지는 행동을 자주 하도록 하고, 줄어드는 행동을 덜 하도록 하여 학습을 진행한다.

구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	• K-최근접 이웃(K-Nearest Neighbor, KNN) • 서포트 벡터 머신(Support Vector Machine, SVM) • 결정 트리(decision tree) • 로지스틱 회귀(logistic regression)
	회귀(regression)	선형 회귀(linear regression)
비지도 학습 (unsupervised learning)	군집(clustering)	• K-평균 군집화(K-means clustering) • 밀도 기반 군집 분석(DBSCAN)
	차원 축소 (dimensionality reduction)	주성분 분석 (Principal Component Analysis, PCA)
강화 학습 (reinforcement learning)	-	마르코프 결정 과정 (Markov Decision Process, MDP)

메뉴바를 클릭하면
계정을 관리할 수 있어요

메뉴바를 클릭하면
계정을 관리할 수 있어요

3. 딥러닝

3-0. 용어 정리

- 딥러닝 : 인간의 뇌를 기초로 하여 설계하였으며, 뉴런과 시냅스의 구성에서 착안하여 병렬 연산을 통해 음성, 영상 인식 등 처리를 가능하게 한다.

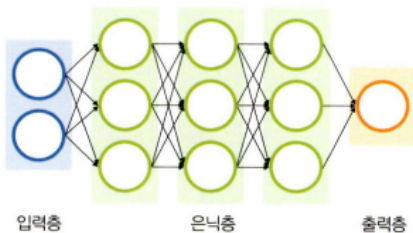
- 심층 신경망을 사용한다는 점이 머신러닝과 큰 차이이며, 여기에는 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐줄 수 있다. 즉, 인간이 하던 작업을 생략하고 대량의 데이터를 신경망에 적용하여 컴퓨터에게 분석을 맡긴다.

- 배치 : 한 번의 학습 단계에서 처리하는 데이터의 묶음 크기이다. 한 배치가 끝나면 가중치가 업데이트 된다.

- 에포크 : 훈련 반복 횟수를 뜻한다.

더보기

- 사전 학습 모델 (pre-trained model) : 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습되어 있는 모델이다.



3-1. 학습 과정

① **데이터 준비** : 파이토치(<https://tutorials.pytorch.kr/>)나 케라스(<http://keras.io/>)를

공하는 데이터셋을 사용하거나 케글의 공개 데이터 역시 사용할 수 있다.

② **모델 정의** : 신경망을 생성하는 단계로, 은닉층의 개수가 많을수록 성능이 좋

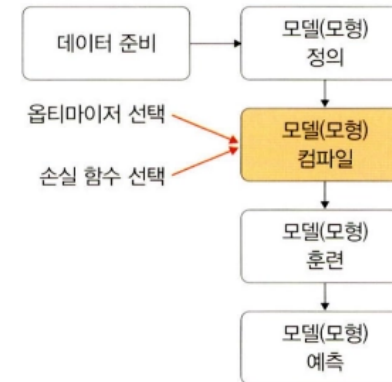
적합이 발생할 가능성이 존재한다.

③ **모델 컴파일** : 활성화 함수, 손실 함수, 옵티마이저를 선택한다.

④ **모델 훈련** : 훈련 단계에서 한 번에 처리할 데이터양을 지정하는 단계이며, 한 번에 처리할 데이터의 영이 많은 경우 학습 속도가 느려지고 메모리 부족 문제가 발생할 수 있기에 배치와 에포크 선택이 중요하다. 또한 훈련 과정에서 값의 변화를 시각적으로 확인하며, 파라미터와 하이퍼파라미터에 대한 최적의 값을 찾을 수 있다.

⑤ **모델 예측** : 검증 데이터셋을 생성한 모델에 적용하여 실제로 예측을 진행한다.

▼ 그림 1-11 딥러닝 모델의 학습 과정



3-2. 학습 알고리즘 : 지도 학습, 비지도 학습, 전이 학습, 강화 학습

- 지도 학습 : 이미지 분석(이미지 분류, 이미지 인식, 이미지 분류)와 시계열 데이터 분석에 사용된다.

- 비지도 학습 : 워드 임베딩(사람의 언어인 자연어를 컴퓨터가 이해할 수 있도록 단어를 벡터로 표현하는 것)과 군집에 사용된다.

- 전이 학습 : 사전 학습 모델을 가지고 우리가 원하는 학습에 미세 조정(fine-tuning) 기법을 이용하여 학습시킨다.

- 강화 학습 : 머신러닝과 동일

공감

메뉴바를 클릭하면
계정을 관리할 수 있어요

Deep Learning

[딥러닝 파이토치 교과서] #02. 파이토치 기초

carchive-lab 2026. 9. 7. 23:15

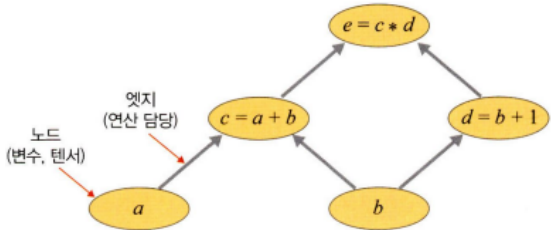
1. 파이토치 개요

더보기

1-0. 용어 설명

- GPU(Graphics Processing Unit) : 연산 속도를 빠르게 하며, 딥러닝에서 GPU 사용은 필수이다.
- 텐서 : 파이토치의 데이터 형태로, 단일 데이터 형식으로 된 자료들의 다차원 행렬이다. 변수 뒤에 `.cuda()`를 추가해서 GPU로 연산을 수행할 수 있다.
- 연산 그래프 : 신경망이 계산을 수행하는 기반으로, 방향성이 있으며 변수를 의미하는 **노드**와 연산을 담당하는 **엣지**로 구성된다.

 : 노드는 변수(a, b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c, d, e)를 구성할 수 있다.



메뉴바를 클릭하면
계정을 관리할 수 있어요

- 동적 신경망 : 훈련을 반복할 때 마다 네트워크 변경이 가능한 신경망이다. 즉, 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능하다.
- 계층(Layer) : 신경망을 이루는 가장 작은 단위 블록
- 모듈 : 하나 이상의 계층들이 모여 특정 기능을 수행하는 단위
- 모델 : 모듈들이 최종적으로 조립되어 하나의 네트워크를 완성한 전체 구조

1-1. 아키텍처 : 파이토치 API + 파이토치 엔진 + 연산 처리

- **파이토치 API** : 실제 계산 수행 없이 파이토치 엔진으로 작업을 전달하는 사용자 인터페이스이다. 다음은 파이토치 API 계층에서 사용자의 편의성을 위해 제공되는 패키지들이다.

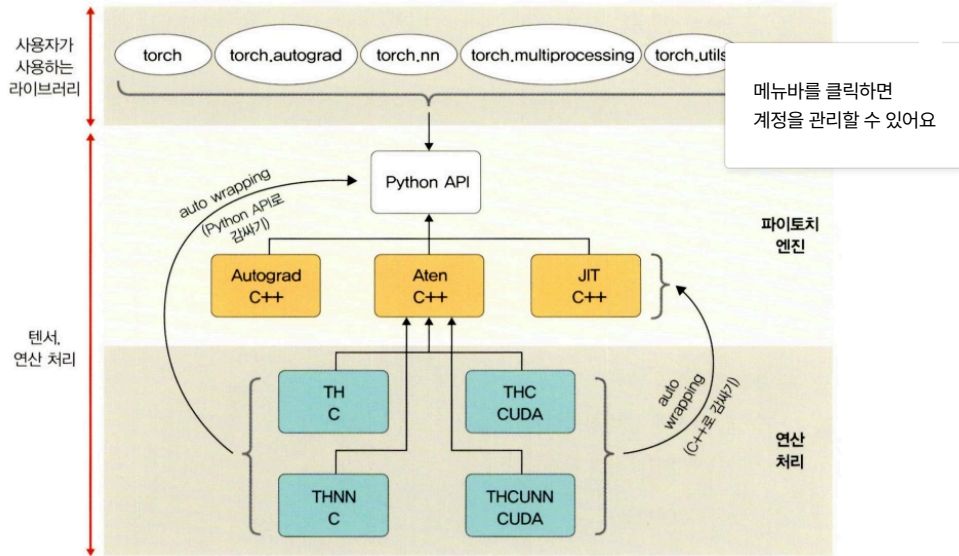
- torch : GPU를 지원하는 텐서 패키지
- torch.autograd : 신경망에 변화를 주더라도 ‘자동 미분’이라는 기술을 통해 연산 그래프 즉시 계산되기 때문에 사용자는 다양한 신경망을 적용할 수 있다.
- torch.nn : 신경망 구축 및 훈련 패키지
- torch.multiprocessing : 파이토치에서 사용하는 프로세스 전반에 걸쳐 텐서의 메모리 공유가 가능하며, 서로 다른 프로세스에서 동일한 텐서에 대한 접근 및 사용이 가능하다.
- torch.utils : 여러 유틸리티 제공하는 패키지

- **파이토치 엔진** (다차원 텐서 및 자동 미분 처리) : Autograd C++, Aten C++, JIT C++, Python API로 구성되며, C++로 감싸 Python API 형태로 제공되기 때문에 사용자들이 손쉽게 모델을 구축하고 텐서 사용을 가능하게 한다.

- Autograd C++ : 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산
- Aten C++ : 텐서 라이브러리 제공
- JIT C++ : 계산을 최적화하기 위한 JIT 컴파일러

- **연산 처리** : C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산을 수행한다.

♥ 그림 2-4 파이토치의 아키텍처

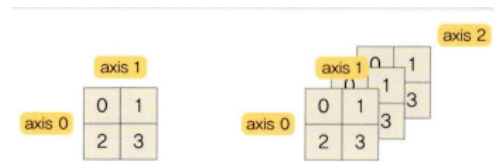


2. 파이토치 기초 문법

2-0. 텐서

- 텐서 (vs 벡터, 행렬)

- 벡터 = 1차원 축(행) = axis 0
- 행렬 = 2차원 축(열) = axis 1
- 텐서 = 3차원 축(채널) = axis2



- 텐서를 메모리에 저장하기 : 텐서는 1차원이든 N차원이든 메모리에 저장할 때는 1차원 배열 형태가 되며, 이 형태여야만 메모리에 저장할 수 있다. 그리고 변환된 1차원 배열을 storage라고 한다.

- 오프셋(offset) : 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스
- 스트라이드(stride) : 각 차원에 따라 다음 요소를 얻기 위해 skip가 필요한 스토리지 요소의 개수. 즉 스트라이드는 메모리에서 텐서 레이아웃을 표현하는 것으로 이해하면 된

다. 요소가 연속적으로 저장되기 때문에 행 중심으로 슬라이드는 항상 1이다.

- 오프셋과 슬라이드는 데이터 자체가 아닌 행렬/텐서를 구분하기 위해 사용한다.

2-1. 텐서 다루기

- 텐서 생성

```
import torch
print(torch.tensor([[1,2],[3,4]])) --- 텐서 생성
print(torch.tensor([[1,2],[3,4]], device="cuda:0")) --- GPU에 생성
```

- 텐서 변환 : ndarray로

```
temp = torch.tensor([[1,2],[3,4]])
print(temp.numpy()) --- 텐서를 ndarray로 변환
```

- 텐서 자료형

- torch.FloatTensor : 32비트의 부동 소수점
- torch.DoubleTensor : 64비트의 부동 소수점
- torch.LongTensor : 64비트의 부호가 있는 정수

- 텐서의 index 조작 : 넘파이의 ndarray 조작하는 것과 유사

```
temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7]) --- 1차원 벡터 생성
print(temp[0], temp[1], temp [-1]) --- 인덱스로 접근
print(temp [2:5], temp [4:-1]) ----- 슬라이스로 접근
```

- 텐서의 차원 조작

- view : 텐서의 차원 형태를 바꾼다

```
temp = torch.tensor([
    [1, 2], [3, 4]])

print(temp.shape)
print(temp.view (4, 1)) --- 2차원 행렬을 4x1 형태의 2차원 열벡터로 재구성 :
[[1], [2], [3], [4]]
print(temp.view(-1)) --- 모든 차원을 풀어서 1차원 벡터로 평탄화 : [1, 2, 3,
```

```
4]
print(temp.view(1,-1)) --- 첫 번째 차원을 1로 고정하고, 나머지 차원은 전체
원소 수(4개)에 맞춰 자동 계산한다. : [[1, 2, 3, 4]]
print(temp.view(-1, 1)) --- 두 번째 차원을 1로 고정하고, 첫 번째 차
동 계산한다 : [[1], [2], [3], [4]]
```

메뉴바를 클릭하면
계정을 관리할 수 있어요

- cat : 서로 다른 길이의 텐서를 특정 축을 따라 하나로 병합
- stack : 새로운 차원을 생성하며 텐서를 결합
- t, transpose : 행렬의 전치 및 차원 순서 변경

- 텐서 연산 : 텐서 간의 타입이 같아야 한다.

2-2. 데이터 준비

- 파일 호출

```
import pandas as pd
import torch
data = pd.read_csv('파일 경로')

x = torch.from_numpy(data['x'].values).unsqueeze(dim=1).float()
y = torch.from_numpy(data['y'].values).unsqueeze(dim=1).float()
```

- 커스텀 데이터셋 : 데이터를 한 번에 다 부르지 않고 조금씩 나누어 부르는 방법

```
import pandas as pd
import torch
from torch.utils.data import Dataset
from torch.utils.data import DataLoader

class CustomDataset(Dataset):
    def __init__(self, csv_file): --- 데이터셋을 부르기
        self.label = pd.read_csv(csv_file)
    def len (self): --- 전체 데이터셋의 크기 반환
        return len(self.label)
    def getitem__(self, idx): --- 전체 x와y 데이터 중에 해당 idx번째의
데이터를 가져오기
        sample = torch.tensor(self.label.iloc[idx, 0:3]).int()
        label = torch.tensor(self.label.iloc[idx,3]).int()
        return sample, label
```

```
tensor_dataset = CustomDataset('파일 경로')
dataset = DataLoader(tensor_dataset, batch_size=4, shuffle=True) --- 배
치 4만큼 가져오기
```

메뉴바를 클릭하면
계정을 관리할 수 있어요

- 이때, DataLoader 객체는 학습에 사용될 데이터 전체를 보관했다가 모델
때 배치 크기만큼 데이터를 꺼내서 사용하는 것으로, 이는 for문을 이용하여 구문을 반
복 실행하는 것과 같다.

- 파이토치 제공하는 데이터셋 : <https://pytorch.org/vision/0.8/datasets>, html 에서 파이토
치에서 제공하는 데이터셋을 확인할 수 있으며, 데이터셋을 내려 받기 위해 http 요청에 대한
처리를 하는 requests 라이브러리를 설치(pip install requests)해야 한다. 데이터셋에는
MNIST, ImageNet를 포함한 유명한 데이터셋을 제공한다.

2-3. 모델 정의

- 단순 정의

```
model = nn.Linear(in_features=1, out_features=1, bias=True)
```

- nn.Module 상속 방식 : 가장 표준적인 방법이며, __init__()와 foward() 함수를 포함한다.

- __init__() : 학습 가능한 가중치를 갖는 계층(레이어)들과 활성화 함수를 미리 선언한
다. super().__init__()을 반드시 호출해 부모 클래스를 초기화해야 한다.
- forward(X): 입력 데이터 X가 들어왔을 때 계층들을 거쳐 어떻게 출력으로 흘러갈지 순
전파 연산 순서를 작성한다.

```
class MLP(Module):
    def __init__(self, inputs):
        super(MLP, self). __init__()
        self.layer = Linear (inputs, 1) --- 계층 정의
        self.activation = Sigmoid() --- 활성화 함수 정의

    def forward(self, X):
        X = self.layer(X)
        X = self.activation(X)
        return X
```

- nn.Sequential 사용 : `__init__()`에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라 `forward()` 함수에서는 모델에서 실행되어야 할 계산을 좀 더 가독성이 뛰어나게 코드로 작성할 수 있다. 이는 모델의 계층이 복잡할수록 효과가 뛰어나다.

메뉴바를 클릭하면
계정을 관리할 수 있어요

```
import torch.nn as nn
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=64,
                      kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=64, out_channels=30,
                      kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.layer3 = nn.Sequential(
            nn.Linear(in_features=30*5*5, out_features=10,
                      bias=True),
            nn.ReLU(inplace=True))

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = x.view(x.shape[0], -1)
        x = self.layer3(x)
        return x

model = MLP()  ----- 모델에 대한 객체 생성
print("Printing children\n-----")
print(list(model.children()))
print("\n\nPrinting Modules\n-----")
print(list(model.modules()))
```

- 함수로 신경망 정의 : Sequential을 이용하는 것과 동일하지만, 변수에 저장해놓은 계층들을 재사용할 수 있다는 장점이 있다.

2-4. 모델의 파라미터 정의 : 손실함수, 옵티마이저, 학습률 스케줄러, 지표

- 손실 함수

- 실제 값과 예측값 사이의 오차를 측정하며, 이 손실 함수의 값을 최소화(global minimum)하는 가중치와 바이어스를 찾는 것이 학습 목표이다.
- global minimum은 오차가 가장 작을 때를 의미하므로 우리가 최종적으로 찾는 것이며, 반대로 local minimum은 global minimum을 찾아가는 과정에서 만나는 hole 같은 것으로 옵티마이저가 여기서 학습을 멈추면 최솟값을 갖는 오차를 찾을 수 없다.
- 종류 : BCELoss(이진분류), CrossEntropyLoss(다중클래스 분류), MESLoss(회귀 모델)

메뉴바를 클릭하면
계정을 관리할 수 있어요

- 옵티마이저

- 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정한다. 이때 모델의 파라미터별로 학습률을 적용시킬 수 있다.
- optimizer는 step() 메서드를 통해 전달받은 파라미터를 업데이트
- torch.optim.Optimizer(params, defaults)는 모든 옵티마이저의 기본이 되는 클래스
- zero_grad() 메서드는 옵티마이저에 사용된 파라미터들의 기울기(gradient)를 0으로 만든다.
- torch.optim.lr_scheduler는 에포크에 따라 학습률을 조절할 수 있다.

- 학습률 스케줄러

- 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소한다.
- 이를 사용하면 초기에는 빠른 학습을 진행하다가 global minimum 근처에 다다르면 학습률을 줄여서 최적점을 찾아갈 수 있도록 한다.

- `optim.lr_scheduler.LambdaLR`: 람다(λ) 함수를 이용하여 그 함수의 결과를 학습률로 설정합니다.
- `optim.lr_scheduler.StepLR`: 특정 단계(step)마다 학습률을 감마(γ) 비율로 감소시킵니다.
- `optim.lr_scheduler.MultiStepLR`: StepLR과 비슷하지만 특정 단계가 아닌 커브에만 감마 비율로 감소시킵니다.
- `optim.lr_scheduler.ExponentialLR`: 에폭마다 이전 학습률에 감마만큼 곱합니다.
- `optim.lr_scheduler.CosineAnnealingLR`: 학습률을 코사인($\cos$) 함수의 형태처럼 변화시킵니다. 따라서 학습률이 커지기도 작아지기도 합니다.
- `optim.lr_scheduler.ReduceLROnPlateau`: 학습이 잘되고 있는지 아닌지에 따라 동적으로 학습률을 변화시킬 수 있습니다.

메뉴바를 클릭하면
계정을 관리할 수 있어요

- 지표 : 훈련과 테스트 단계를

2-5. 모델 훈련

- ① 기울기 초기화 : 파이토치에서는 기울기가 누적되기 때문에 `optimizer.zero_grad()`로 초기화 해야 한다.
- ② 기울기 자동 계산 : `loss.backward()`는 배치가 반복될 때마다 오차가 중첩적으로 쌓이기 때문에, 1번의 방법으로 미분 값을 0으로 초기화한다.

```
for epoch in range(100):
    yhat = model(x_train)
    loss = criterion(yhat, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

2-6. 모델 평가 : 함수 또는 모듈 이용 (혹은 사이킷런에서 제공하는 혼동 행렬 이용)

- 가장 먼저 **pip install torchmetrics**

- 함수 이용

```
import torch
import torchmetrics

preds = torch.randn(10, 5).softmax(dim=-1)
target = torch.randint(5, (10,))
acc = torchmetrics.functional.accuracy(preds, target)
```

- 모듈 이용

```
import torch
import torchmetrics

metric = torchmetrics.Accuracy() --- 모델 평가(정확도) 초기화

n_batches = 10
for i in range(n_batches):
    preds = torch.randn(10, 5).softmax(dim=-1)
    target = torch.randint(5, (10,))

    acc = metric(preds, target)
    print(f"Accuracy on batch {i}: {acc}") --- 현재 배치에서
    모델 평가(정확도)
acc = metric.compute()
print(f"Accuracy on all data: {acc}") --- 모든 배치에서 모델 평가(정확도)
```

메뉴바를 클릭하면
계정을 관리할 수 있어요

2-7. 훈련 과정 모니터링 : 텐서 보드 이용

① 가장 먼저 **pip install tensorboard**

② 텐서 보드를 위한 코드 작성

```
import torch
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("../chap02/tensorboard")

for epoch in range(num_epochs):
    model.train() --- 학습 모드로 전환(dropout True)
    batch_loss = 0.0
    for i, (x, y) in enumerate(dataloader):
        x, y = x.to(device).float(), y.to(device).float()
        outputs = model(x)
        loss = criterion(outputs, y)
        writer.add_scalar("Loss", loss, epoch) --- 스칼라 값(오차)을 기록

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

```
writer.close() --- SummaryWriter가 더 이상 필요하지 않으면 close() 메서드 호출
```

③ 텐서 보드 실행 **tensorboard --logdir=./chap02/tensorboard --port=6006**

④ 웹 브라우저에서 **http://localhost:6006**으로 실행

메뉴바를 클릭하면 계정을 관리할 수 있어요

공감

'Deep Learning' 카테고리의 다른 글		
[딥러닝 파이토치 교과서] #01. 머신러닝과 딥러닝	(0)	2026.09.07

'Deep Learning'의 다른글

이전글 [딥러닝 파이토치 교과서] #01. 머신러닝과 딥러닝

현재글 : [딥러닝 파이토치 교과서] #02. 파이토치 기초

관련글

carchive-lab 님의 블로그

carchive-lab 님의 블로그 입니다.

댓글 0

carchive-lab

내용을 입력하세요.

등록

메뉴바를 클릭하면 계정을 관리할 수 있어요

carchive-lab 님의 블로그

carchive-lab 님의 블로그 입니다.

글쓰기 블로그 관리

분류 전체보기 (2) 

Deep Learning (2) 

Tag

머신러닝 #딥러닝

최근글 인기글

[딥러닝 파이토치 교과서] #02. 파이토치 기초

2026.09.07 23:15

[딥러닝 파이토치 교과서] #01. 머신러닝과 딥러닝

2026.09.07 21:53

최근댓글